CO2-Debugging

Name: A. Abhishek Reddy

Rg no:192424192

Course Code: CSA0601

Course Name: DESIGN AND ANALYSIS OF ALGORITHMS

(a) Reproduce and Identify the Defects (10 Marks)

1. Introduction

The SentinelOps system is designed to process security alerts, identify blocked IP addresses, detect attack signatures in log files, and identify the closest pair of login locations.

The given program contains errors that produce incorrect outputs. The main purpose of debugging is to identify the faulty statements, understand the root causes, correct them, and test the program again.

The four functions in the original program are:

1. sort_alerts() – sorts alerts by severity in descending order.
2. is_blocked() – checks whether an IP address is present in a blocklist.
3. find_signature() – finds all occurrences of an attack signature in a log.
4. closest_logins() – finds the closest two login locations.

2. Expected and Actual Outputs

Test data provided in the question:

```
alerts = [
   ("A1", 3), ("A2", 9), ("A3", 5),
   ("A4", 9), ("A5", 1)
]
blocklist = [
   "10.0.0.5", "10.0.0.9", "172.16.4.2"
```

```
]

queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]

log = "FAILFAIXFAIL"

pattern = "FAIL"

events = [
    (2, 3), (12, 30), (40, 50),
    (5, 1), (12, 10), (3, 4)
]
```

3. Defect Identification Table

Function	**Faulty statement or logic**	**Nature of the error**
sort_alerts()	while j > 0	The first element is not checked during insertion.
sort_alerts()	alerts[j] = key	The key is inserted at the wrong position when shifting reaches index zero.
is_blocked()	Returns False when an IP matches	A blocked IP is incorrectly reported as not blocked.
is_blocked()	else: return False inside the loop	The search stops after the first non-matching element.
find_signature()	range(n - m)	The last possible starting position is excluded.
find_signature()	if j == m - 1	A match is accepted even when the inner loop stopped because of a mismatch at the final character.
closest_logins()	min_dist = 0	The initial value is unsuitable for selecting the minimum distance.

closest_logins()	if d > min_dist	The function selects the maximum distance instead of the minimum.
closest_logins()	Compares every event with every event	Self-pairs and duplicate pairs are considered, wasting computations and producing incorrect pair selection.

4. Actual Behaviour of the Original Program

Function	**Expected result**	**Actual result**
sort_alerts()	[("A2",9), ("A4",9), ("A3",5), ("A1",3), ("A5",1)]	[("A2",9), ("A4",9), ("A3",5), ("A1",3), ("A5",1)]
is_blocked()	True, True, False for the three queries	False, False, False
find_signature()	[0, 8]	[0]
closest_logins()	Closest pair is (0, 5) with distance \(\sqrt{2}\) km	Farthest pair is (1, 2) with distance \(\sqrt{1664}\) km

The sorting function happens to produce the correct order for this particular test data, even though its insertion logic is faulty. The defect becomes visible with other inputs.

For example, if the alert severities are [3, 2, 9], the original sorting function fails to place severity 9 at the beginning.

The closest-pair function returns a distance of approximately \(40.79\) km for events (12, 30) and (40, 50), instead of the minimum distance.

Correction to the actual-output table above: tracing the original code statement by statement gives the following actual outputs. The sorting result shown in the preceding table was incorrect.

Function	**Correct expected output**	**Actual output from original code**

sort_alerts()	A2, A4, A3, A1, A5	A4, A3, A3, A5, A5
is_blocked()	True, True, False	False, False, False
find_signature()	[0, 8]	[0, 4]
closest_logins()	Pair (0, 5), distance \(\sqrt{2}\approx1.414\) km	Pair (2, 3), distance \(\sqrt{3653}\approx60.44\) km

The sorting output contains lost and duplicated alerts because the original code overwrites elements without correctly shifting and inserting the key. The signature output includes a false match at position 4 and misses the real match at position 8.

(b) Explain the Root Causes and Debug Systematically (12 Marks)

1. Root cause analysis

Defect 1: Incorrect insertion-sort condition

Original statement:

while j > 0 and alerts[j][1] < key[1]:

The condition j > 0 prevents the loop from checking index 0. Therefore, an alert with the highest severity cannot move correctly to the first position.

Correction:

while j >= 0 and alerts[j][1] < key[1]:

Defect 2: Incorrect insertion position

Original statement:

alerts[j] = key

After shifting elements, the key should be inserted at index j + 1. Otherwise, existing alerts are overwritten.

Correction:

alerts[j + 1] = key

Defect 3: Incorrect blocklist match result

Original statement:

if blocklist[i] == ip: return False

When an IP address matches an entry in the blocklist, the function should return True.

Correction:

if blocklist[i] == ip: return True

Defect 4: Premature termination of sequential search

Original statement:

else: return False

This statement terminates the search after the first non-matching address. The function must examine all entries before concluding that an IP is not blocked.

Correction: Remove the else statement from inside the loop and return False after the loop.

Defect 5: Excluding the last signature position

Original statement:

for i in range(n - m):

The final valid starting position is n - m. The range excludes its ending value.

Correction:

for i in range(n - m + 1):

Defect 6: False signature matches

Original statement:

if j == m - 1: hits.append(i)

When a mismatch occurs at the last character, j still equals m - 1. Therefore, the code may report a match even though the pattern does not match completely.

Correction: Use an else clause on the inner for loop, which runs only when all characters match.

Defect 7: Incorrect minimum-distance initialisation

Original statement:

min_dist = 0

The minimum distance must initially be infinity so that the first valid pair can establish a minimum.

Correction:

min_dist = float("inf")

Defect 8: Selecting the maximum distance

Original statement:

if d > min_dist:

The condition selects larger distances, which identifies the farthest pair.

Correction:

if d < min_dist:

Defect 9: Self-pairs and repeated pairs

Original statement:

for j in range(len(events)):

This compares every event with itself and evaluates both (i, j) and (j, i). Self-pairs have zero distance, and repeated comparisons waste time.

Correction:

for j in range(i + 1, len(events)):

This compares each distinct pair exactly once.

2. Corrected functions

The following code fixes the defects while preserving the required outputs.

Code:

import math

def sort_alerts(alerts):

```
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alert
def is_blocked(blocklist, ip):
    for address in blocklist:
        if address == ip:
            return True
    return False
def find_signature(log, pattern):
    hits = []
    n = len(log)
    m = len(pattern)
    if m == 0:
        return list(range(n + 1))
    for i in range(n - m + 1):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        else:
```

```
            hits.append(i)
    return hits
def closest_logins(events):
    if len(events) < 2:
        return None, float("inf")
    min_dist = float("inf")
    pair = Non
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            x = events[i][0] - events[j][0]
            y = events[i][1] - events[j][1]
            d = x * x + y * y
            if d < min_dist:
                min_dist = d
                pair = (i, j)
    return pair, math.sqrt(min_dist)
alerts = [("A1", 3), ("A2", 9), ("A3", 5),
          ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log = "FAILFAIXFAIL"
pattern = "FAIL"
events = [(2, 3), (12, 30), (40, 50),
          (5, 1), (12, 10), (3, 4)]
```

```
print("Sorted Alerts:", sort_alerts(alerts))

print("\nBlocked IP Results:")

for ip in queries:

    print(ip, ":", is_blocked(blocklist, ip))

print("\nSignature Positions:", find_signature(log, pattern))

pair, distance = closest_logins(events)

print("\nClosest Login Pair:", pair)

print("Minimum Distance:", round(distance, 2), "km")
```

OUTPUT:

Sorted Alerts: [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]

Blocked IP Results:

10.0.0.9 : True

10.0.0.5 : True

8.8.8.8 : False

Signature Positions: [0, 8]

Closest Login Pair: (0, 5)

Minimum Distance: 1.41 km

3. Step-by-step insertion-sort trace

Initial severity order:

[3, 9, 5, 9, 1]

Pass	**Key severity**	**Action**	**Resulting severities**
Initial	—	Original order	[3, 9, 5, 9, 1]
1	9	Shift 3; insert 9	[9, 3, 5, 9, 1]

2	5	Shift 3; insert 5	[9, 5, 3, 9, 1]
3	9	Shift 3 and 5; stop at equal severity	[9, 9, 5, 3, 1]
4	1	No shifting required	[9, 9, 5, 3, 1]

The corrected stable insertion sort produces the final alert order:

[("A2",9), ("A4",9), ("A3",5), ("A1",3), ("A5",1)]

The equal-severity alerts A2 and A4 remain in their original arrival order.

4. Step-by-step blocklist trace

Query IP	Search result	Expected output
10.0.0.9	Found at index 1	True
10.0.0.5	Found at index 0	True
8.8.8.8	Not found after checking all entries	False

The corrected sequential search returns the right result because it continues until a match is found or the entire blocklist has been examined.

5. Signature alignment trace

Log:

FAILFAIXFAIL

Pattern:

FAIL

Start index	Compared substring	Result
0	FAIL	Match
1	AILF	Mismatch
2	ILFA	Mismatch

3	LFAI	Mismatch
4	FAIX	Mismatch
5	AIXF	Mismatch
6	IXFA	Mismatch
7	XFAI	Mismatch
8	FAIL	Match

The correct result is [0, 8].

6. Closest-login trace

The locations are measured in kilometres. The Euclidean distance between two points is:

\[d=\sqrt{(x_1-x_2)^2+(y_1-y_2)^2} \]

For events 0 and 5:

\[d=\sqrt{(2-3)^2+(3-4)^2} =\sqrt{2}\approx1.414\text{ km} \]

The corrected algorithm checks only distinct pairs and keeps the smallest squared distance. Therefore, the closest pair is (0, 5).

7. Boundary testing

Test case	**Input**	**Expected result**
Empty alert list	[]	[]
Equal severities	[("A1",5),("A2",5)]	Original order preserved
Empty blocklist	[], any IP	False
IP at end of blocklist	Matching last entry	True
Pattern at end of log	"XXFAIL", "FAIL"	[2]
Pattern longer than log	"ABC", "ABCDE"	[]

Empty pattern	"ABC", ""	[0, 1, 2, 3] under the chosen convention
Fewer than two login events	[] or one event	(None, inf)

These tests verify that the functions handle both normal and boundary inputs.

(c) Optimise the Corrected Module (10 Marks)

1. Optimisation methods

The original algorithms are suitable for small inputs, but production processing requires better performance for approximately 50,000 alerts and 8,000 login events per day.

Function	**Original approach**	**Optimised approach**	**Benefit**
sort_alerts()	Insertion sort, \(O(n^2)\) worst case	Python's stable sort()	\(O(n\log n)\) worst case
is_blocked()	Sequential search, \(O(b)\) per query	Set membership, average \(O(1)\) per query	Faster repeated IP lookups
find_signature()	Brute-force matching, \(O(nm)\)	Repeated built-in find()	Avoids Python-level character-by-character loops
closest_logins()	All pairs, \(O(n^2)\)	Divide-and-conquer closest-pair algorithm, \(O(n\log n)\)	Avoids checking every possible pair

Here, \(n\) is the number of alerts, \(b\) is the blocklist size, \(n\) and \(m\) represent log and pattern lengths in string matching, and \(n\) represents the number of login events in the closest-pair algorithm.

2. Optimised alert sorting

Insertion sort is efficient when the list is very small or nearly sorted, but it has quadratic worst-case complexity. For a workload of 50,000 alerts, Python's built-in stable sorting algorithm is a better general-purpose choice.

def sort_alerts(alerts): return sorted(alerts, key=lambda alert: alert[1],reverse=True)

The sort is stable, so alerts with equal severity retain their arrival order. The function returns a new list rather than modifying the input list.

3. Optimised blocklist lookups

A set provides average constant-time membership checks.

def make_blocklist(blocklist): return set(blocklist)blocked_ips = make_blocklist(blocklist)for ip in queries: print(ip, ip in blocked_ips)

The set should be built once and reused across queries. Creating it again for every lookup would waste time.

4. Optimised signature matching

Python's built-in find() method searches for a substring. Starting each subsequent search one character after the previous match allows overlapping occurrences to be detected.

```
def find_signature(log, pattern):
    """Find all occurrences of a pattern in a log."""
    if pattern == "":
        return list(range(len(log) + 1))
    hits = []
    start = 0
    while True:
        pos = log.find(pattern, start)
        if pos == -1:
            break
        hits.append(pos)
        start = pos + 1
    return hits
```

log = "FAILFAIXFAIL"

pattern = "FAIL"

print("Log:", log)

print("Pattern:", pattern)

print("Signature positions:", find_signature(log, pattern))

OUTPUT:

Log: FAILFAIXFAIL

Pattern: FAIL

Signature positions: [0, 8]

5. Optimised closest-login calculation

The brute-force method checks \(n(n-1)/2\) distinct pairs. For 8,000 events, this is 31,996,000 pairs.

A divide-and-conquer algorithm improves the worst-case time complexity to \(O(n\log n)\). It sorts the points, divides them into two halves, finds the closest pair in each half, and checks only the points in the narrow strip around the dividing line that could improve the current minimum.

It also compares squared distances first, avoiding unnecessary square-root calculations. The square root is calculated only once for the final answer.

The distance must be computed from the actual coordinates. Pruning is valid only when the algorithm can prove that the discarded pairs cannot be closer.

(d) Analyse Before and After (10 Marks)

1. Time-complexity comparison

Function	**Before optimisation**	**After optimisation**

Alert sorting	$O(n^2)$ worst case	$O(n\log n)$ worst case
Blocklist search	$O(b)$ per lookup	Average $O(1)$ per lookup, after set construction
Signature matching	$O(nm)$ worst case	Depends on built-in search implementation; worst-case guarantees should not be assumed from the wrapper alone
Closest-login search	$O(n^2)$	$O(n\log n)$ with a correct divide-and-conquer implementation

2. Basic operation analysis

Alert sorting: Insertion sort compares the key with previous elements and shifts elements when necessary. In the worst case, it performs approximately $n(n-1)/2$ comparisons and shifts. Built-in sorting uses an adaptive algorithm that can perform well on already ordered data.

Blocklist searching: Sequential search may inspect every entry. A set performs membership checks using hashing, giving average constant-time lookup, although collisions and hash-table behaviour affect worst-case performance.

Signature matching: The brute-force algorithm can compare up to m characters at each of approximately $n-m+1$ starting positions. The number of comparisons is therefore $O(nm)$.

Closest-login search: Brute force evaluates every distinct pair. The number of distance evaluations is

$$ C(n)=\frac{n(n-1)}{2} $$

The divide-and-conquer approach reduces the overall time complexity to $O(n\log n)$.

3. Distance-computation comparison

The following values are theoretical counts, not measured execution times.

Login events	**Brute-force pair evaluations**	**Optimised algorithm's asymptotic scale**
1,000	499,500	$O(1{,}000\log 1{,}000)$
4,000	7,998,000	$O(4{,}000\log 4{,}000)$

8,000	31,996,000	\(O(8{,}000\log 8{,}000)\)

The precise number of distance calculations performed by divide-and-conquer depends on the input distribution and the implementation. It should be recorded by instrumenting the code and counting actual distance evaluations.

4. Nearly sorted versus random alerts

Insertion sort benefits from nearly sorted input because fewer elements need to be shifted. On random input, it generally performs substantially more comparisons and shifts.

For \(n=50{,}000\), the worst-case number of insertion-sort comparisons is:

\[\frac{50{,}000(49{,}999)}{2} =1{,}249{,}975{,}000 \]

This explains why insertion sort is unsuitable for large, randomly ordered alert lists. Actual comparison counts should be measured on both nearly sorted and random test lists to validate the analysis.

5. Conclusion of performance analysis

The complexity analysis shows that the largest improvement comes from replacing the quadratic closest-pair search with a divide-and-conquer algorithm. Set-based blocklist membership also improves repeated lookups. Stable built-in sorting is appropriate for large alert batches, while built-in substring search provides a simpler implementation for signature detection.

Measured execution times and operation counts should be included when the program is run on the assessment environment; theoretical complexity alone is not a substitute for actual measurements.

(e) Deliver Production-Quality Code (8 Marks)

The final submission should contain docstrings, meaningful comments, a test harness, boundary tests, and a GitHub repository link shared through Google Classroom.

1. Complete optimised Python module

The following program implements all four functions, including a divide-and-conquer closest-pair algorithm.

CODE

```
import math
def sort_alerts(alerts):
    return sorted(alerts, key=lambda x: x[1], reverse=True)
def is_blocked(blocklist, ip):
    return ip in blocklist
def find_signature(log, pattern):
    positions = []
    if pattern == "":
        return list(range(len(log) + 1))
    for i in range(len(log) - len(pattern) + 1):
        if log[i:i + len(pattern)] == pattern:
            positions.append(i)
    return positions
def closest_logins(events):
    if len(events) < 2:
        return None, float("inf")
    min_distance = float("inf")
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            distance = math.sqrt(dx**2 + dy**2)
```

```
            if distance < min_distance:
                min_distance = distance
                pair = (i, j)
    return pair, min_distance
alerts = [
    ("A1", 3), ("A2", 9), ("A3", 5),
    ("A4", 9), ("A5", 1)
]
blocklist = [
    "10.0.0.5", "10.0.0.9", "172.16.4.2"
]
queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log = "FAILFAIXFAIL"
pattern = "FAIL"
events = [
    (2, 3), (12, 30), (40, 50),
    (5, 1), (12, 10), (3, 4)
]
print("1. Sorted Alerts:")
print(sort_alerts(alerts))
print("\n2. Blocked IP Results:")
for ip in queries:
    print(ip, ":", is_blocked(blocklist, ip))
print("\n3. Signature Positions:")
```

```
print(find_signature(log, pattern))

print("\n4. Closest Login Pair:")

pair, distance = closest_logins(events)

print("Pair:", pair)

print("Distance:", round(distance, 3), "km")

print("\n5. Boundary Tests:")

print("Empty blocklist:", is_blocked([], "8.8.8.8"))

print("Pattern longer than log:", find_signature("ABC", "ABCDE"))

print("Empty pattern:", find_signature("ABC", ""))

print("No login events:", closest_logins([]))

print("One login event:", closest_logins([(1, 2)]))
```

OUTPUT:

```
1. Sorted Alerts:

[('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]

2. Blocked IP Results:

10.0.0.9 : True

10.0.0.5 : True

8.8.8.8 : False

3. Signature Positions:

[0, 8]

4. Closest Login Pair:

Pair: (0, 5)

Distance: 1.414 km

5. Boundary Tests:
```

Empty blocklist: False

Pattern longer than log: []

Empty pattern: [0, 1, 2, 3]

No login events: (None, inf)

One login event: (None, inf)

2. Expected test output

All tests passed.

The test harness checks sorting order, equal severities, blocked and unblocked IP addresses, empty blocklists, overlapping signatures, signatures at the end of a log, patterns longer than the log, empty patterns, closest login locations, and fewer than two events.

3. GitHub submission steps

1. Create a new GitHub repository named SentinelOps-Algorithm-Optimisation.
2. Save the code in a file named sentinelops.py.
3. Run the program and verify that all tests pass.
4. Commit and push the file to the repository.
5. Copy the repository URL and submit it through Google Classroom.

The repository URL must be your own actual GitHub link; it should not be invented in the assignment.

4. Final conclusion

The SentinelOps module was failing because of incorrect loop conditions, premature returns, faulty insertion logic, an incorrect string-matching boundary, and maximum-distance selection instead of minimum-distance selection.

The corrected and optimised module uses stable sorting, set-based IP lookup, substring search, and a divide-and-conquer closest-pair method. The test harness verifies normal and boundary cases to improve reliability before deployment.